Phase 18.2 — Project & Task Domain Foundation

SUMMARY

Phase 18.1.8 is VERIFIED and must be treated as the stable Agent execution baseline.

MANDATORY REPOSITORY INSPECTION

Before changing code: Read `AGENTS.md`; Read the Phase 18.1.8 report: `.ai/context/phase-18.1.8-structured-agent-contract.md`; Inspect all current domain/application infrastructure related to AgentRun, AgentDefinition, Artifact, Workspace, project-like identifiers if any already exist, correlation IDs, repositories, API DTOs, migrations, operational metrics; Search the entire repository for `Project`, `Task`, `Dependency`, `Artifact`, `Workspace`, `AgentRun`; Reuse existing abstractions where they already satisfy the requirement; Do not create duplicate concepts.

SCOPE

Introduce the minimum persistent domain required to represent a software project and its executable work items. The core model should support: Project -> Tasks -> Dependencies. Future AgentRuns must be able to belong to a Project and, where appropriate, a Task. Do not implement Planner behavior yet. Do not implement Coding behavior yet.

PROJECT DOMAIN

Introduce a persistent Project entity with fields for Id, stable project identifier, Name, Description, Status, created/updated timestamps, optional repository/workspace reference if the existing architecture supports it cleanly. Use repository conventions for IDs and timestamps. Do not add speculative fields for billing, users/teams, deployment environments, GitHub organizations, AI model configuration, project budgets.

PROJECT LIFECYCLE

Define an explicit Project status model supporting newly created project, active project, paused project, completed project, cancelled/archived project. Use explicit transition validation rather than arbitrary string assignment. Keep lifecycle rules centralized in the domain/application layer according to repository conventions.

TASK DOMAIN

Introduce a persistent Task entity belonging to a Project with Id, ProjectId, stable task identifier, Title, Description, Status, task type/category, priority, created/updated timestamps, optional AgentRun association where appropriate. Task types should be generic enough for future workflow stages. A single Task model must support future roles.

TASK LIFECYCLE

Define explicit task statuses: pending/ready, running, blocked, succeeded, failed, cancelled. Choose names consistent with existing conventions. Centralize valid transitions. Lifecycle must support determining if a task is executable now without invoking an LLM. Do not implement the scheduler yet.

TASK DEPENDENCIES

Introduce dependency relationship between Tasks: Task B depends on Task A. Task B is blocked until predecessor is completed successfully. Reject self-dependency, duplicate dependency, cross-project dependency. Deleting a Task must

not silently create invalid dependency records. Circular dependency detection must exist at domain/application boundary.

PROJECT ↔ AGENTRUN RELATIONSHIP

Integrate AgentRun with Project without breaking existing Agent functionality. Prefer nullable relationships for backward compatibility. Pre-existing AgentRun must continue to work. Where a run belongs to a Task: Project -> Task -> AgentRun. Do not force every existing AgentRun to belong to a Project. Do not change existing AgentRun lifecycle semantics or listed runtime/claiming/result contract components unless minimally unavoidable.

PERSISTENCE

Use EF Core and SQLite conventions already established. If migrations are required: create migration, inspect generated migration, verify no unexpected destructive operations, test clean migration, test migration from current latest schema, test preservation of existing AgentRun/AgentDefinition/AgentMemory/Artifact data. Existing data must survive unchanged. Do not introduce new database, event sourcing, or second persistence mechanism.

REPOSITORIES / APPLICATION SERVICES

Introduce repositories and application services only where they provide real separation from persistence. Project capabilities: create, get by ID, list, update, lifecycle transition. Task capabilities: create, get, list by project, update, lifecycle transition, dependency management, determine dependency satisfaction. Avoid generic CRUD abstractions merely for abstraction's sake.

DEPENDENCY READINESS

Introduce deterministic application-level query/service answering whether a Task can execute now based only on persisted project/task/dependency state. Examples: no dependencies -> ready; dependency succeeded -> ready; dependency pending/running/failed/cancelled -> blocked. Do not invoke Hermes or an LLM.

API

Expose minimal APIs for Projects (create/get/list/update/status transition), Tasks (create/get/list by project/update/status transition), Dependencies (add/remove/inspect dependencies/readiness). Use DTOs separate from domain entities. Do not expose EF entities directly. Do not add UI. Do not synchronously execute agents from endpoints.

AGENTRUN API COMPATIBILITY

Existing AgentRun endpoints must behave exactly as before. If ProjectId/TaskId are added to AgentRun DTOs, keep nullable. Do not expose internal runtime/secret fields. Preserve existing structuredResult behavior from Phase 18.1.8.

METRICS

Do not add speculative metrics. If needed, add only minimum useful project/task counters or lifecycle info. Do not redesign observability.

TESTS

Add deterministic tests for Project (creation, retrieval, update, valid/invalid lifecycle transitions, not-found), Task (creation, ownership, retrieval/listing, valid/invalid lifecycle transitions), Dependencies (valid dependency, duplicate/self/cross-project/circular rejection, deletion behavior), Readiness (ready/blocked cases by dependency states), AgentRun integration (existing valid without Project/Task, Project-linked persists, Task-linked persists, structured result intact), API (new endpoints/DTO behavior, deterministic errors for invalid operations).

REAL HERMES

Do not add new real-Hermes execution solely for Project/Task CRUD proof. Real Hermes was already verified in Phase 18.1.8. Only run existing Hermes regression if implementation materially changes AgentRun/Orchestrator execution behavior. Avoid unnecessary LLM calls.

SECURITY

Project/Task data must not automatically grant additional agent capabilities. Do not introduce unrestricted filesystem access, shell execution, Git execution, deployment, credentials, arbitrary network access.

EXPLICIT NON-GOALS

Do NOT implement Planner Agent, LLM-based task decomposition, automatic DAG generation, Specification Agent, Architecture Agent, Coding Agent, Test Agent, Reviewer Agent, repair loop, Git automation, OpenViking, multi-agent orchestration, autonomous commits, autonomous deployment, learning/experience system, UI/dashboard.

VALIDATION GATE

Run targeted Project tests, Task tests, dependency/readiness tests, AgentRun integration tests, API tests, full deterministic regression, Debug build, Release build, migration tests if applicable. Do not repeat full real-Hermes stability suite unless AgentRun execution code materially changed. Never fabricate test evidence.

DOCUMENTATION

Create `.ai/context/phase-18.2-project-task-domain.md` documenting final Project model, final Task model, lifecycle state machines, dependency rules, readiness semantics, AgentRun relationships, migrations, API, tests, compatibility considerations, known limitations, next-phase recommendation.

DEFINITION OF DONE

Phase is GO only when Project and Task domains exist and persist; lifecycles validated; dependencies persisted and invalid dependencies rejected; circular dependencies rejected; deterministic readiness exists; AgentRun optionally belongs to Project/Task; existing AgentRun behavior compatible; structured AgentResult from 18.1.8 intact; migration data-preserving; API tested; regression passes; Debug and Release builds have 0 warnings/errors; no unnecessary future architecture introduced. Final report must state `Project/Task Domain: VERIFIED / NOT VERIFIED` and `Final Decision: GO / NO-GO`. Do not commit or push unless explicitly instructed. Do not claim unexecuted evidence.